

Random Forest : Atelier

Réalisé par: Salma El Angui

I. Travaux pratiques à réaliser

- Importer et charger le jeu de données choisi pour votre mini-projet.
- Effectuer les étapes nécessaires de préparation des données : nettoyage, encodage, normalisation (si nécessaire) puis division en ensembles d'entraînement et de test
- Former **deux modèles Random Forest** à partir des données prétraitées : un modèle pour la **régression**, un modèle pour la **classification**
- Évaluer les performances de chacun des deux modèles en calculant les métriques appropriées (ex. : accuracy, F1-score, RMSE...).
- Procéder à des ajustements si nécessaire
- Enregistrer les modèles entraînés

Model 1: Classification

Concernant le modèle de Random Forest Classification, nous avons choisi le dataset `data_set_meteo` afin de prédire la présence ou l'absence de pluie.

L'objectif est d'appliquer l'algorithme de classification Random Forest pour déterminer si les conditions météorologiques entraînent ou non des précipitations.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder

|
df = pd.read_csv('dataset_meteo.csv', sep=',')
encoder = LabelEncoder()
df['Pluie_binaire'] = encoder.fit_transform(df['pluie'])
```

j'ai utilisé **LabelEncoder** pour convertir la variable catégorielle "pluie" en une variable binaire numérique.

Cela permet au modèle Random Forest de traiter correctement l'information.

Le texte ("oui"/"non") est automatiquement transformé en valeurs 0 et 1.

```
X = df[['Temperature_C', 'Humidite_%', 'Vent_kmh', 'Pression_hPa']]
y = df['Pluie_binaire']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Créer Le modèle
rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)

# Entraîner
rf_clf.fit(X_train, y_train)

# Test du modèle
y_pred = rf_clf.predict(X_test)

# Evaluation
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle : {acc:.2f}")
```

Accuracy du modèle : 0.87

Model 2: Regression

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv('Student_Performance.csv', sep=',')
encoder = LabelEncoder()
df['Extracurricular_Activities_binary'] = encoder.fit_transform(df['Extracurricular_Activities'])
df['Extracurricular_Activities_binary']

#features
X = df[['Hours Studied', 'Previous Scores', 'Extracurricular_Activities_binary',
        'Sleep Hours', 'Sample Question Papers Practiced']]
y = df['Performance Index']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Créer Le modèle
rf_reg = RandomForestRegressor(n_estimators=100, max_depth=4, min_samples_leaf=300, random_state=42)

# Entraîner
rf_reg.fit(X_train, y_train)

# Test du modèle
y_pred = rf_reg.predict(X_test)

# Évaluation du modèle
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error : {mse:.2f}")
print(f"R^2 Score : {r2:.2f}")
```

Mean Squared Error : 24.46
R^2 Score : 0.93

II. Questions de réflexion — Réponses



1. Quel modèle offre la meilleure précision entre Bagging et Random Forest ? Justifiez.

En général, le modèle **Random Forest** est plus précis que le simple Bagging.

Même si les deux techniques reposent sur l'idée de combiner plusieurs arbres pour obtenir un modèle plus stable. Random Forest est une extension du Bagging qui ajoute une couche supplémentaire de randomisation en sélectionnant aléatoirement un sous-ensemble de variables à chaque division d'arbre.

Cette petite randomisation supplémentaire rend les arbres beaucoup plus différents les uns des autres, ce qui réduit les erreurs communes et améliore la capacité du modèle à généraliser.

2. En quoi Random Forest améliore-t-il le simple Bagging ?

Random Forest améliore le Bagging en ajoutant une sélection aléatoire de variables lors de la construction des arbres. Cela rend chaque arbre plus indépendant, diminue l'overfitting et améliore la performance globale.

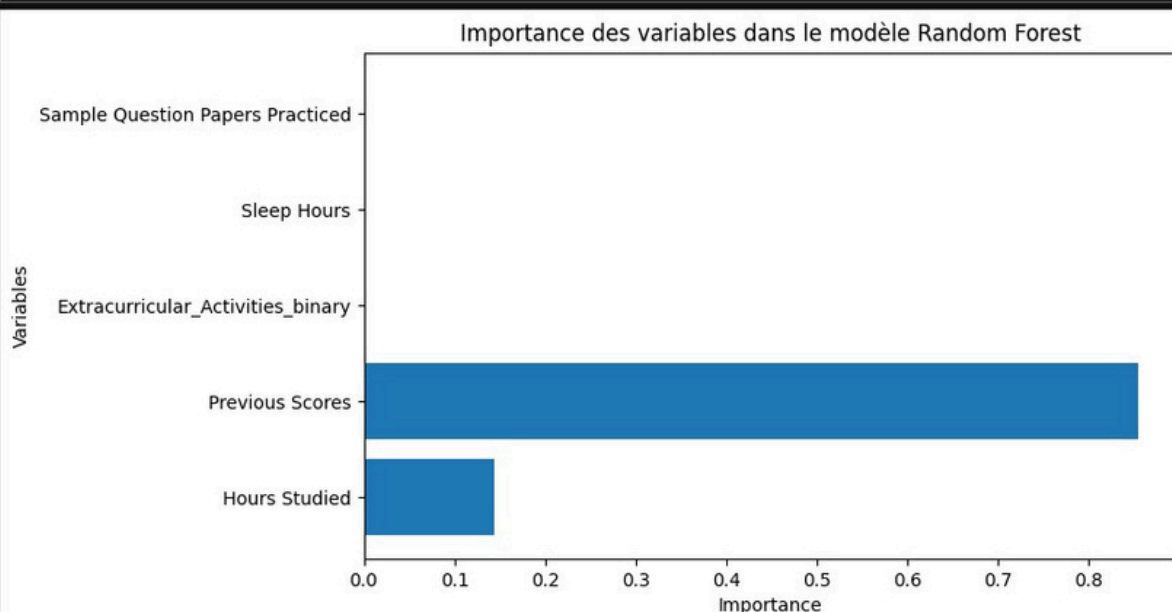
3. Visualiser l'importance des variables (feature_importances_)

Regression

```
import matplotlib.pyplot as plt

# Récupérer les importances
importances = rf_reg.feature_importances_
features = X.columns

# Visualisation
plt.figure(figsize=(8, 5))
plt.barh(features, importances)
plt.xlabel("Importance")
plt.ylabel("Variables")
plt.title("Importance des variables dans le modèle Random Forest")
plt.show()
```

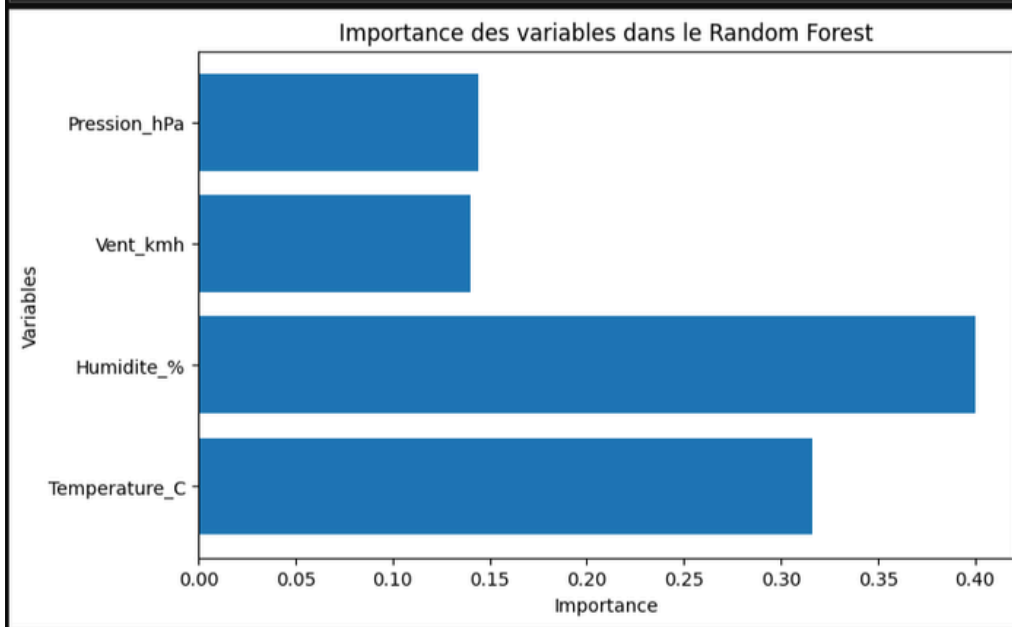


Classification

```
import matplotlib.pyplot as plt

importances = rf_clf.feature_importances_
features = X.columns

plt.figure(figsize=(8,5))
plt.barh(features, importances)
plt.xlabel("Importance")
plt.ylabel("Variables")
plt.title("Importance des variables dans le Random Forest")
plt.show()
```



Après visualisation des *feature_importances_*, on observe une différence notable entre la classification et la régression.

En classification, toutes les variables présentent une certaine importance, car chacune contribue à la séparation des classes,

En revanche, en régression, l'objectif est de prédire une valeur continue. Le modèle retient donc principalement les variables qui expliquent réellement la variance de la cible.

4.Modifier le nombre d'estimateurs (10, 100, 500) et observer l'impact

```

: # Tester plusieurs nombres d'estimateurs
for n in [10, 100, 500]:
    print(f"\n Test avec n_estimators = {n} ")

    rf_clf = RandomForestClassifier(n_estimators=n, random_state=42)
    rf_clf.fit(X_train, y_train)

    y_pred = rf_clf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    print(f"Accuracy = {acc:.3f}")

```

Test avec n_estimators = 10
Accuracy = 0.874

Test avec n_estimators = 100
Accuracy = 0.870

Test avec n_estimators = 500
Accuracy = 0.870

```

R2 Score : 0.934

# Tester plusieurs nombres d'estimateurs
for n in [10, 100, 500]:
    print(f"\n Test avec n_estimators = {n} ")

    rf_reg = RandomForestRegressor(n_estimators=n, max_depth=4, min_samples_leaf=300, random_state=42)
    rf_reg.fit(X_train, y_train)

    y_pred = rf_reg.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    print(f"MSE= {mse:.3f}")
    print(f"R2= {r2:.3f}")

```

Test avec n_estimators = 10
MSE= 26.277
R2= 0.929

Test avec n_estimators = 100
MSE= 24.455
R2= 0.934

Test avec n_estimators = 500
MSE= 24.239
R2= 0.934

Plus on ajoute d'arbres, plus la prédiction devient stable.

Les performances s'améliorent un peu entre 10 et 100 arbres, puis se stabilisent à partir de 100 arbres.